The first thing you should know about tries, is that it's actually pronounced "try". Its author[8], René de la Briandais, chose this term because it was similar to tree, but also because it’s part of the word re**trie**val, which is the main purpose of this container; then its peculiar pronunciation is partly meant as an indented pun, and partly meant to avoid confusion with "tree".

Tries were originally developed as a compact, efficient way to search strings in files; the idea behind this data structure is, as we have seen in the previous section, providing a way to reduce redundancy by storing common prefixes of strings just once.

This couldn’t be achieved using a plain binary search tree, or in general with just a binary tree, so a paradigm-shift was needed: de la Briandais then used n-ary trees, where edges are marked with all the characters in an alphabet, and nodes just connects paths.

Well, to be thorough, nodes also have a small but crucial function as well: they store a tiny bit of information stating if the path from root to current node corresponds to a key stored in the trie.

Let’s take a look at figure 6.2 before moving to a more formal description: it shows the structure of a typical trie, containing the words “`a`”, “`an`”, “`at`” and “`I`”.

If you feel that figure 6.2 is a bit confusing… you are right, it is: in their classic implementation, trie’s nodes have one edge for each possible character in the alphabet used: some of these edges point to other nodes, most of them